

Java Coding Questions

1. **String reverse.** Write a method that will take one string as an argument and return the reverse version of this string.

```
public class ReverseStr {  
    public static void main(String[] args) {  
        System.out.println(revStr("apple")); // elppa  
        System.out.println(revStr("John")); // nhoJ  
        System.out.println(revStr("phone")); // enohp  
        System.out.println(revStr("1234567")); // 7654321  
    }  
  
    // Big O(n) linear  
    public static String revStr(String str) {  
        StringBuilder res = new StringBuilder();  
        for (int i = str.length() - 1; i >= 0; i--) {  
            res.append(str.charAt(i));  
        }  
        return res.toString();  
    }  
}
```

2. Array reverse. Write a method that will take an array as an argument and reverse it.

```
import java.util.Arrays;

public class ReverseArray {
    public static void main(String[] args) {
        int[] inputArr = new int[] {1, 2, 3, 4, 5};
        // [1, 2, 3, 4, 5]
        System.out.println(Arrays.toString(inputArr));
        revArr(inputArr);
        // [5, 4, 3, 2, 1]
        System.out.println(Arrays.toString(inputArr));
    }
    // Big O(n) linear
    public static void revArr(int[] arr) {
        int start = 0;
        int end = arr.length - 1;
        while (start < end) {
            int tmp = arr[start];
            arr[start] = arr[end];
            arr[end] = tmp;
            start++;
            end--;
        }
    }
}
```

3. Reverse words. Write a method that will take a string as an argument. The method will reverse the position of words and return it.

```
public class ReverseWords {  
    public static void main(String[] args) {  
        System.out.println(revWords("apple banana kiwi"));  
        // kiwi banana apple  
  
        System.out.println(revWords("I am John Doe"));  
        // Doe John am I  
  
        System.out.println(revWords("orange"));  
        // orange  
    }  
  
    // Big O(n) linear  
    public static String revWords(String str) {  
        StringBuilder res = new StringBuilder();  
        String[] words = str.split(" ");  
        for (int i = words.length - 1; i >= 0; i--) {  
            res.append(words[i]).append(" ");  
        }  
        return res.toString().trim();  
    }  
}
```

4. Prime number. A prime number is a number that can only be divided by itself and 1 without remainder.

```
public class PrimeNumber {  
    public static void main(String[] args) {  
        System.out.println(isPrime(29)); // true  
        System.out.println(isPrime(35)); // false  
        System.out.println(isPrime(20)); // false  
        System.out.println(isPrime(7)); // true  
    }  
  
    /**  
     * Big O(n) linear  
     * A prime number is a number that can only be divided by itself and  
     * 1 without remainder.  
     */  
    public static boolean isPrime(int num) {  
        // create numbers from 2 till half inclusive of the num  
        for (int i = 2; i <= num / 2; i++) {  
            // try each number by using %  
            if (num % i == 0) {  
                return false;  
            }  
        }  
        return true;  
    }  
}
```

5. String palindrome. A palindrome is a word, phrase, number, or sequence of words that reads the same backward as forward.

```
public class StringPalindrome {  
    public static void main(String[] args) {  
        System.out.println(isPal("anna")); // true  
        System.out.println(isPal("apple")); // false  
        System.out.println(isPal("level")); // true  
    }  
  
    /**  
     * Big O(n) linear  
     * A palindrome is a word, phrase, number, or sequence of words that  
     reads the same backward as forward.  
     */  
    public static boolean isPal(String str) {  
        int start = 0;  
        int end = str.length() - 1;  
        while (start < end) {  
            // if pointers values are not equal return false  
            if(str.charAt(start) != str.charAt(end)){  
                return false;  
            }  
            start++;  
            end--;  
        }  
        return true;  
    }  
}
```

6. Number palindrome. A palindrome is a word, phrase, number, or sequence of words that reads the same backward as forward. The straightforward solution would be to convert the number to a string and use the above approach. Some interviewers will not allow it. So let's take a look at what we can do here.

```
public class NumberPalindrome {
    public static void main(String[] args) {
        System.out.println(isPal(545)); // true
        System.out.println(isPal(13)); // false
        System.out.println(isPal(33)); // true
    }
    /**
     * Big O(n) linear
     */
    public static boolean isPal(int num) {
        // 1. To get the most right number, we can do 'right = num % 10'
        // 2. To remove right number, we can do 'num = num / 10'
        // 3. To concatenate to the number, we can do 'num = (num * 10) +
singleDigitNumber'
        int copy = num;
        int rev = 0;
        int rightMost;
        while (num > 0) {
            rightMost = num % 10; // get right most
            rev = (rev * 10) + rightMost; // concatenate to rev
            num = num / 10; // remove right most from num
        }
        return copy == rev;
    }
}
```

7. Max/min number from an array. Write a method that will accept an array of int as an argument and it returns the max/min number from a given array.

```
public class MaxNumber {  
    public static void main(String[] args) {  
        System.out.println(max(new int[] {4, 781, 8, 99, 103})); // 781  
        System.out.println(max(new int[] {1, 2, 3, 4, 5})); // 5  
        System.out.println(max(new int[] {3, 4})); // 4  
        System.out.println(max(new int[] {100})); // 100  
    }  
  
    /**  
     * Big O(n) linear  
     */  
    public static int max(int[] arr) {  
        int max = arr[0]; // assume first element is biggest  
  
        // check assumption  
        for (int num : arr) {  
            // if max wasn't the biggest number, then update it  
            if (max < num) {  
                max = num;  
            }  
        }  
  
        return max;  
    }  
}
```

```

public class MinNumber {
    public static void main(String[] args) {
        System.out.println(min(new int[] {4, 781, 8, 99, 103})); // 4
        System.out.println(min(new int[] {1, 2, 3, 4, 5})); // 1
        System.out.println(min(new int[] {3, 4})); // 3
        System.out.println(min(new int[] {100})); // 100
    }

    public static int min(int[] arrNum) {
        // assume first element of array is the smallest number
        int min = arrNum[0];

        // loop over the array and test assumption
        for (int num : arrNum) {
            // if min was not smallest, update it
            if (min > num) {
                min = num;
            }
        }

        return min;
    }
}

```


8. Find the second min/max number from an array. Write a method that accepts an int array as an argument and returns a **second** or **n** min/max number from the given array.

```
/**
 * Big O(n) linear
 * secMax({4, 781, 8, 99, 103}) -> 103
 * secMax({1, 2, 3, 4, 5})      -> 4
 * secMax({3, 4})              -> 3
 */
public static int secMax(int[] arr) {
    // 1. Get max/min and save index
    int maxIndex = getMaxIndex(arr, -1);

    // 2. Get max/min by avoiding first max
    return arr[getMaxIndex(arr, maxIndex)];
}

private static int getMaxIndex(int[] arr, int avoidIndex) {
    int max = Integer.MIN_VALUE;
    int maxIndex = -1;
    for (int i = 0; i < arr.length; i++) {
        if (i != avoidIndex && max < arr[i]) {
            max = arr[i];
            maxIndex = i;
        }
    }
    return maxIndex;
}
```

```

/**
 * Big  $O(n \log n)$ 
 * secMin({4, 781, 8, 99, 103}) -> 8
 * secMin({1, 3, 2, 4, 5})      -> 2
 * secMin({3, 4})               -> 4
 */
public static int secMin(int[] numArr) {
    Arrays.sort(numArr);
    if (numArr.length > 1) {
        return numArr[1];
    }
    return numArr[0];
}

/**
 * Big  $O(n \log n)$ 
 * Return n biggest value from array.
 *
 * nMax([4, 3, 2, 8, 9, 5], 1); -> 9
 * nMax([4, 3, 2, 8, 9, 5], 2); -> 8
 * nMax([4, 3, 2, 8, 9, 5], 3); -> 5
 * nMax([4, 3, 2, 8, 9, 5], 100); -> 0
 */
public static int nMax(int[] arr, int n) {
    if (arr.length < n) {
        return 0;
    }

    Arrays.sort(arr);
    return arr[arr.length - n];
}

```

9. String Pool and == operator to compare references in Java. Let's see an example. What's the output of this program? Why this output?

```
public class StringPoolAndEquals {  
    public static void main(String[] args) {  
        String str = "apple";  
        String str1 = "apple";  
        String str2 = new String("apple");  
  
        // == checks if two references pointing to the same object or not  
        System.out.println(str == str1); // true  
        System.out.println(str == str2); // false  
        System.out.println(str.equals(str2)); // true  
    }  
}
```

10. Swap values of two variables without direct reassignment and without creating any extra variables.

```
public class Swap {  
    public static void main(String[] args) {  
        int j = 15;  
        int i = 10;  
  
        // TODO swap values of i and j without creating any variables  
        j = j - i; // j = 15 - 10; j = 5  
        i = i + j; // i = 10 + 5; i = 15  
        j = i - j; // j = 15 - 5; j = 10  
  
        System.out.println(i); // 15  
        System.out.println(j); // 10  
    }  
}
```

11. Two-string anagram. An anagram is when all the letters in one string exist in another but the order of letters does not matter. Write a method that accepts two string arguments and returns true if they are anagram and false if they are not.

```
import java.util.Arrays;

public class TwoStringAnagram {
    public static void main(String[] args) {
        System.out.println(isAnagram("listen", "silent")); // true
        System.out.println(isAnagram("triangle", "integral")); // true
        System.out.println(isAnagram("abc", "bca")); // true
        System.out.println(isAnagram("abc", "ccb")); // false
        System.out.println(isAnagram("aaa", "aaab")); // false
    }
    /**
     * Big  $O(n \log(n))$ 
     * An anagram is when all the letters in one string exist in another
     * but the order of letters does not matter.
     */
    public static boolean isAnagram(String str, String str1) {
        // convert both strings to char[]
        char[] arrStr = str.toCharArray();
        char[] arrStr1 = str1.toCharArray();
        // sort both char[] arrays
        Arrays.sort(arrStr);
        Arrays.sort(arrStr1);
        // compare sorted char[]
        return Arrays.equals(arrStr, arrStr1);
    }
}
```

12. Remove duplicates from a string. Write a method that accepts one string argument and returns it without duplicates. We will see two versions of this method.

```
import java.util.LinkedHashSet;
import java.util.Set;

public class RemoveDuplicatesFromStr {
    public static void main(String[] args) {
        System.out.println(removeDup("hello")); // helo
        System.out.println(removeDup("applep")); // aple
        System.out.println(removeDup("aaaaaa")); // a
        System.out.println(removeDup("abc")); // abc
    }

    /**
     * Big O(n) linear
     * removeDup("hello") -> helo
     * removeDup("applep") -> aple
     * removeDup("aaaaaa") -> a
     * removeDup("abc") -> abc
     */
    public static String removeDup(String str) {
        StringBuilder strNoDup = new StringBuilder();
        for (char ch : str.toCharArray()) {
            if (!strNoDup.toString().contains(String.valueOf(ch))) {
                strNoDup.append(ch);
            }
        }
        return strNoDup.toString();
    }
}
```

```
// Big O (n) linear
// or do it with Set
public static String removeDupWithSet(String str) {
    // load to set to remove dup
    Set<Character> set = new LinkedHashSet<>();
    for (char ch : str.toCharArray()) {
        set.add(ch);
    }

    // convert back to string
    StringBuilder strNoDup = new StringBuilder();
    for (char ch : set) {
        strNoDup.append(ch);
    }
    return strNoDup.toString();
}
}
```

13. Count letters(Map). Write a method that accepts a string as an argument. The method counts the number of appearances of each char and returns a map. The key will be a letter and the value will be the number of appearances in the string. See the input and output in the example.

```
import java.util.LinkedHashMap;
import java.util.Map;

public class CountLetters {
    public static void main(String[] args) {
        System.out.println(countLetters("hello")); // {h=1, e=1, l=2, o=1}
        System.out.println(countLetters("aaaaaa")); // {a=6}
        System.out.println(countLetters("abc")); // {a=1, b=1, c=1}
        System.out.println(countLetters("abca")); // {a=1, b=1, c=1}
    }
    /**
     * Find out how many of each letter exist in the string
     */
    public static Map<Character, Integer> countLetters(String str) {
        Map<Character, Integer> letters = new LinkedHashMap<>();
        for (char ch : str.toCharArray()) {
            if (letters.containsKey(ch)) {
                int count = letters.get(ch);
                letters.put(ch, count + 1);
            } else {
                letters.put(ch, 1);
            }
        }
        return letters;
    }
}
```


14. **FizzBuzz**. Print numbers from 1 to 100

- If a number is divisible by 3 print **Fizz**
- If a number is divisible by 5 print **Buzz**
- If a number is divisible by both 3 and 5 print **FizzBuzz**

```
public class FizzBuzz {  
    public static void main(String[] args) {  
        fizzBuzz();  
    }  
    /**  
     * Big O(1) constant  
     * Print numbers from 1 to 100 inclusive, but  
     * - if number is divisible by 3, print Fizz  
     * - if number is divisible by 5, print Buzz  
     * - if number is divisible by 3 and 5, print FizzBuzz  
     */  
    public static void fizzBuzz() {  
        for (int i = 1; i <= 100; i++) {  
            if (i % 3 == 0 && i % 5 == 0) {  
                System.out.println("FizzBuzz");  
            } else if (i % 5 == 0) {  
                System.out.println("Buzz");  
            } else if (i % 3 == 0) {  
                System.out.println("Fizz");  
            } else {  
                System.out.println(i);  
            }  
        }  
    }  
}
```

15. **Even or Odd.** Write a method that will accept one int as an argument. The method prints ***Even*** if the number is even and ***Odd*** if the number is odd.

```
public class EvenOrOdd {  
    public static void main(String[] args) {  
        evenOrOdd(5); // Odd  
        evenOrOdd(2); // Even  
        evenOrOdd(100); // Even  
        evenOrOdd(101); // Odd  
    }  
  
    // Big O(1) constant  
    public static void evenOrOdd(int num) {  
        if (num % 2 == 0) {  
            System.out.println("Even");  
        } else {  
            System.out.println("Odd");  
        }  
    }  
}
```

16. Sum of two. Write a method that accepts an `int[]` array and an `int` number, and find 2 elements in the array that sum is equal to the given `int`. Assume that an input array will have only one pair of numbers that sum equal to our given number. It will always have this pair. See input and output examples. I use the Brute Force algorithm.

```
import java.util.Arrays;

public class SumOfTwo {
    /**
     * Big  $O(n^2)$  quadratic
     * sum([1, 2, 3, 5], 4)      -> [1, 3]
     * sum([7, 7, 4, 3, 8], 7)  -> [4, 3]
     * sum([13, 43, 2, 71], 84) -> [13, 71]
     * sum([1, 2, 3, 4], 100)   -> [0, 0]
     */
    public static int[] sumOfTwo(int[] arr, int target) {
        int[] res = new int[2];

        for (int i = 0; i < arr.length; i++) {
            for (int j = i + 1; j < arr.length; j++) {
                if (arr[i] + arr[j] == target) {
                    res[0] = arr[i];
                    res[1] = arr[j];
                    return res;
                }
            }
        }

        return res;
    }
}
```

17. The Fibonacci. It is a series of numbers where the next number is the sum of the previous two numbers. The first two numbers of the Fibonacci is 0 followed by 1. Write a method that will accept one int number *n*. The method will print *n* number of Fibonacci numbers.

```
public class Fibonacci {  
    public static void main(String[] args) {  
        fib(3); // 0 1 1  
        fib(5); // 0 1 1 2 3  
        fib(10); // 0 1 1 2 3 5 8 13 21 34  
    }  
    /**  
     * The Fibonacci is a series of numbers where the next number is the  
sum of the previous two numbers.  
     * The first two numbers of the Fibonacci is 0 followed by 1.  
     * Print n Fibonacci numbers in one line separated by space  
     */  
    public static void fib(int n) {  
        int numOne = 0;  
        int numTwo = 1;  
        System.out.print(numOne + " ");  
        System.out.print(numTwo + " ");  
        for (int i = 0; i < n - 2; i++) {  
            int sum = numOne + numTwo;  
            System.out.print(sum + " ");  
            numOne = numTwo;  
            numTwo = sum;  
        }  
        System.out.println();  
    }  
}
```

18. Balanced String. This question has become very popular lately. The program should find out if an input string is balanced or not.

```
import java.util.Stack;

public class BalancedStr {
    public static void main(String[] args) {
        System.out.println(isBalanced("[{()}]")); // true
        System.out.println(isBalanced("[({})]")); // false
        System.out.println(isBalanced("{[]}")); // false
        System.out.println(isBalanced("({}{}([{}]))")); // true
        System.out.println(isBalanced("{()}")); // false
    }

    public static boolean isBalanced(String str) {
        Stack<Character> st = new Stack<>();
        for (char ch : str.toCharArray()) {
            if (ch == '{' || ch == '[' || ch == '(') {
                st.push(ch);
            } else {
                if (st.isEmpty()) { return false; }
                char lOpen = st.pop();
                if (ch == '}' && lOpen != '{') { return false; }
                else if (ch == ']' && lOpen != '[') { return false; }
                else if (ch == ')' && lOpen != '(') { return false; }
            }
        }

        return st.isEmpty();
    }
}
```

19. Sort array without built-in sort methods. In this example, we will use selection sort and bubble sort algorithms to sort our array.

```
import java.util.Arrays;

public class SelectionSort {
    public static void main(String[] args) {
        int[] arrNum = {6, 5, 2, 1, 9, 10, 0};
        System.out.println(Arrays.toString(arrNum)); // [6, 5, 2, 1, 9, 10, 0]
        sSort(arrNum);
        System.out.println(Arrays.toString(arrNum)); // [0, 1, 2, 5, 6, 9, 10]
    }
    /**
     * Selection Sort - Big O(n^2) quadratic
     */
    public static void sSort(int[] arr) {
        for (int i = 0; i < arr.length; i++) {
            int minIndex = i;
            int min = arr[i];
            for (int j = i + 1; j < arr.length; j++) {
                if (min > arr[j]) {
                    min = arr[j];
                    minIndex = j;
                }
            }
            int tmp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = tmp;
        }
    }
}
```

```

import java.util.Arrays;

public class BubbleSort {
    public static void main(String[] args) {
        int[] arrNum = {6, 5, 2, 1, 9, 10, 0};
        System.out.println(Arrays.toString(arrNum)); // [6, 5, 2, 1, 9, 10, 0]
        bSort(arrNum);
        System.out.println(Arrays.toString(arrNum)); // [0, 1, 2, 5, 6, 9, 10]
    }

    /**
     * Bubble Sort - Big O(n^2) quadratic
     */
    public static void bSort(int[] arr) {
        int swapCounter = -1;
        int unsortedArrLength = arr.length;
        while (swapCounter != 0) {
            swapCounter = 0;
            for (int i = 0; i < unsortedArrLength - 1; i++) {
                if (arr[i] > arr[i + 1]) {
                    int tmp = arr[i + 1];
                    arr[i + 1] = arr[i];
                    arr[i] = tmp;
                    swapCounter++;
                }
            }
            unsortedArrLength--;
        }
    }
}

```